

sol3_research_agent_sdk. White paper

A topic in. An evidence-backed technical white paper out.

Claude Agent SDK. Seven roles. Python owns the phases.

This is **not** the old config-only port. PRs #157 / #158 grew it into a report generator. Saturday Lab 3 still fills two functions. This folder writes `paper.md`.

Spillwave Solutions | spillwave.com

What you will build (already filled)

ARTIFACT	PATH
the paper	work/<slug>/paper.md
figures	work/<slug>/diagrams/*.png
knowledge bundle	work/<slug>/knowledge/research/
checkpoint	work/<slug>/.harness/state.json

Cast lives in `roleplan.py`. Pipeline lives in `paper.py`. One PreToolUse hook in `roles.py`. Checks in `checks.py`. Exits in `gates.py`.

Why this folder exists

The same loop, in a different runtime. The rubric, the red gate, the write scope, and the exits did not have to change to make it run.

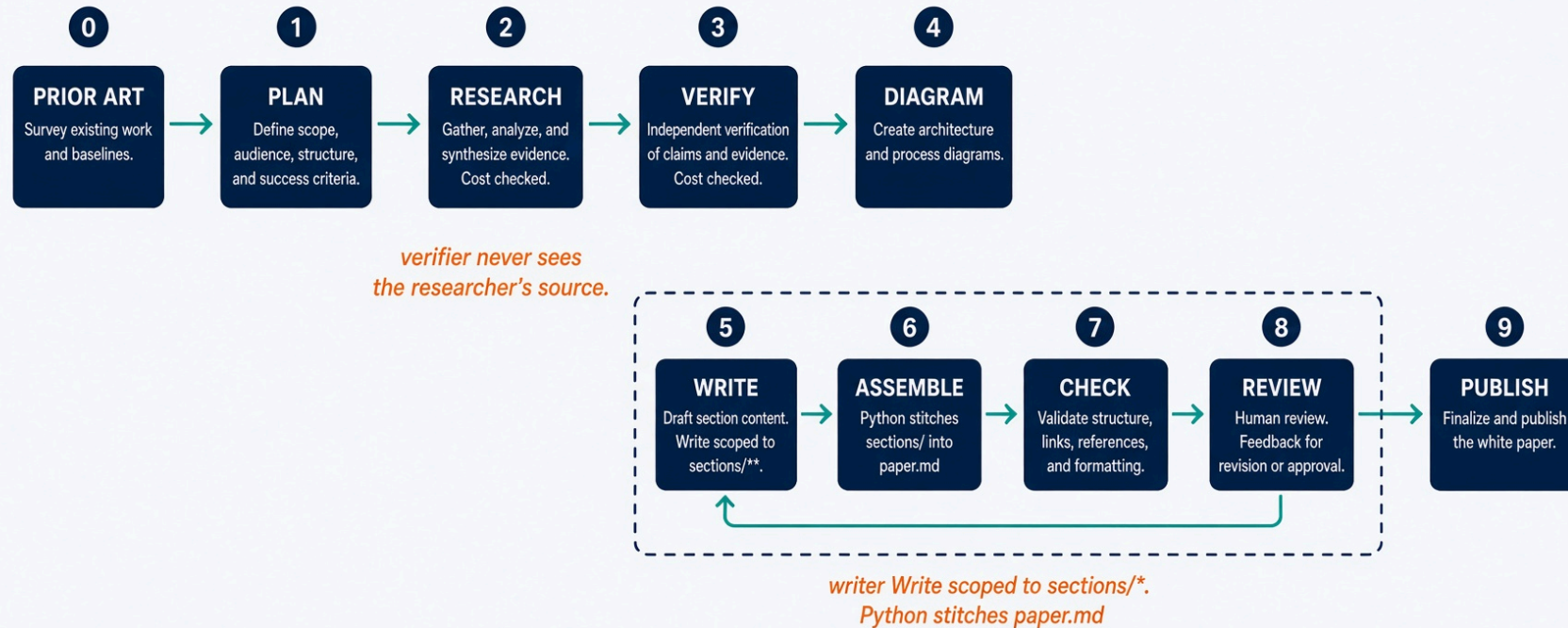
A searcher that can write can edit the evidence to fit the paper. A verifier shown the researcher's answer is not a second opinion. A model handed Bash widens the blast radius from "a wrong paper" to "anything this machine can run".

Learning objectives

- Print the seven-role table with no SDK installed
- Explain why `allowed_tools` is a session-wide allowlist
- Declare MCP in Python, not via a gitignored `.mcp.json`
- Cap questions, claims, and dollars as three separate budgets
- Check cost **inside** research and verify, not only at the gate
- Refuse to publish a paper that did not pass

Starting architecture

Agent SDK white paper. Ten phases. Python owns the order.



Three exits: pass / retry / escalate. Cost checked inside research and verify, not only at the gate.

The cast. Seven roles, one writer.

orchestrator	Task	writes nothing
planner	Read, Glob, Grep	returns plan JSON. Python writes plan.json
researcher	Read + search MCP	writes nothing
verifier	Read + search MCP	writes nothing. Never sees the source.
diagrammer	Read, Glob, Grep	returns diagram source. Python renders.
writer	Read + Write	sections/** only. Cannot reach paper.md
judge	Read, Glob, Grep	writes nothing

task table. Judge, researcher, verifier must print `no` in the writes column.
If any prints `yes`, stop.

Two places enforce scope. You need both.

```
tools=[...]      decides whether a role can write at all
PreToolUse hook  decides which paths it may write
```

One hook serves the whole cast. It reads `agent_type` off the tool call.

sol1 registered one hook per writing role. That does not generalize: several hooks on `write` all run, an empty dict means "no opinion", and the first role that shrugs lets another role's write through.

A write with no `agent_type` is denied. So is a write from anyone but the writer. So is a write to `paper.md`, `claims.json`, or outside the work dir.

`allowed_tools` is not the parent's list

It is a session-wide permission allowlist. It gates subagents too.

First live run: options said `allowed_tools=["Agent"]` with `permission_mode="dontAsk"`. Researcher held Perplexity, Context7, WebSearch. Every call came back denied.

What the researcher then did is the part worth keeping. It did **not** answer from memory. It reported "NO RESEARCH WAS PERFORMED", named each denied tool, and returned an empty claim list. The run escalated with "no source produced a single claim" rather than shipping four fabricated sections.

`options_for` now allows the union of what the cast holds. `Bash` is denied at the top. The parent is kept from writing by the hook, not the allowlist.

MCP is declared in Python

```
def mcp_servers() -> dict:
    servers = {"context7": {"type": "http", "url": "https://mcp.context7.com/mcp"}}
    key = os.environ.get("PERPLEXITY_API_KEY")
    if key:
        servers["perplexity-ask"] = {...}
    return servers
```

`strict_mcp_config=True`. `.mcp.json` holds a key, so it is gitignored. A fresh clone has no such file. The researcher then loses both servers with **no error**, and a topic it cannot search reads as a topic nobody has written about.

Perplexity is included only when the key is set. An empty key is an auth error on every question, which is the same failure wearing a different hat.

Ten phases. Python owns the order.

0 prior_art	second brain, skip if missing	-> prior-art.md
1 plan	sections and questions	-> plan.json
2 research	claims from primary sources	-> sources.json, claims.json
3 verify	independent second look	-> verdicts.json
4 diagram	source in, Python renders	-> diagrams.json
5 write	one section at a time	-> sections/*.md
6 assemble	stitch + references, in Python	-> paper.md
7 check	seven deterministic rows	-> check.json
8 review	judge on what a script cannot	-> review.json
9 publish	private gist, on request	-> gist.json

Phases 0 to 4 run once. 5 to 8 are the retry cycle. Re-running research because a paragraph lost its citation marker buys a bill, not a better paper.

The planner is told the budget

```
--max-questions default 12. --max-diagrams default 4.
```

A live planner returned seven sections and twenty-eight questions. Truncating to four questions left a paper with two sections and five orphaned headings.

A planner that knows the ceiling writes a whole paper under it. Python still enforces the cap afterwards.

Verify is independent. Claims are capped.

The verifier is given the claim text and nothing else. A claim the two disagree on is `disputed`. One the verifier cannot reach is `unverified`, never silently dropped.

Four questions produced 111 claims on one live run. Every claim is a verification turn.

`--max-claims` default 24. Claims with a number, a version, or a date are checked first. The rest stay `unverified`. The writer states them qualitatively. The knowledge bundle records that nobody checked them.

Cost is checked inside the phases

`--max-usd` default 5.00. Checked inside research, inside verification, and at the gate.

Checking only at the gate is a bug this port had: the gate runs once per attempt, and a twenty-four question research phase can spend the whole budget several times over before the gate ever sees it.

Running out mid-verification leaves remaining claims `unverified`. Marking them verified because the money ran out is the lie.

Seven deterministic rows. No model vote.

ROW	PASS WHEN
complete	every section the plan named is in the paper
grounded	every [n] resolves to a retrieved source
cited	every claim paragraph names a source
sourced	every arXiv / DOI / author-year appears in retrieved text
images	every figure the paper references is a file on disk
style	zero em dashes outside code spans
(plus sources)	the reference list is intact

complete looks redundant. A run that spent its budget in verification wrote no sections at all: title, abstract, 25-entry reference list. Every other row passed. The judge caught it. The row is there so the first backstop does not have to.

The check that matters most: `sourced`

A web search cannot refute a citation that was never published. Asking a model whether a reference is real gets you a confident yes.

The only thing that catches a fabricated arXiv id or DOI is looking for it in the text that was actually retrieved. `checks.ungrounded_identifiers`.

Three exits. Stall is the one people miss.

`gates.py`: pass, retry, escalate. No fourth.

When an attempt fails in exactly the same way as the last one, the loop is not converging. The signature is **what** failed, not how it was worded. A model rephrasing its own complaint does not read as progress.

A paper that did not pass is never published.

Commands. Offline first.

```
cd solutions/sol3_research_agent_sdk
task table          # no SDK, no key, no network
task checks         # checks / gates / publish / rkc --demo
task test           # unit tests, same constraints
task setup          # SDK + imagen-diagrams clone into .cache/
task demo           # full offline run over the fixture
task run TOPIC="how MCP servers authenticate"
task publish TOPIC="..." # also pushes a private gist
```

Expected output

```
work/<slug>/  
  paper.md  
  diagrams/*.png  
  knowledge/research/      RKC: sources, claims, evidence, findings  
  .harness/state.json      per-phase status and cost
```

Deleting one phase output re-runs that phase and no other. Resume needs no bookkeeping: a phase whose file exists already ran.

`--private` on a gist means unlisted, not access controlled. Anyone holding the URL can read the paper and fetch every figure. Treat the URL as the credential.

Plugin files vs Python

```
plugin/agents/research-{{planner,researcher,verifier,diagrammer,writer,judge}}.md  
plugin/skills/research-loop/SKILL.md
```

The skill is the readable specification of what `paper.py` implements. Do **not** run the skill from this port. Python owns the phases. Running the skill would give you two orchestrators disagreeing about whose turn it is.

When an agent markdown file and the role table disagree, `options_for` raises.

Troubleshooting

SYMPTOM	CAUSE	FIX
Subagent tools denied	<code>allowed_tools</code> too narrow	union of the cast, plus hook
Silent empty research	inherited missing <code>.mcp.json</code>	<code>mcp_servers()</code> in Python
Fabricated sections	researcher answered from memory	empty claims must escalate
Bill before a gate	cost only at the gate	check inside research and verify
Title plus references, no body	spent budget in verify	<code>complete</code> row, then judge
Writer rewrote <code>paper.md</code>	hook not looking at <code>agent_type</code>	deny anyone but writer, deny that path

Validation checklist

- ☐ `task table`: one writer, everyone else `no`
- ☐ `task test` needs no SDK, no key, no network
- ☐ Verifier is never handed the researcher's source
- ☐ Writer cannot reach `paper.md` or `claims.json`
- ☐ A write with no `agent_type` is denied
- ☐ `maxTurns` not `max_turns` (TypeError on the real SDK)
- ☐ A paper that did not pass is never published

Spillwave Solutions | spillwave.com

Recap

What we built. A topic in, a paper plus a knowledge bundle out.

Takeaways

1. Trigger and runtime change. Exits do not.
2. `allowed_tools` gates the children.
3. MCP in Python, or a fresh clone silently cannot search.
4. Three budgets, and cost inside the phases.
5. `sourced` catches a citation a model will bless.

Closing line. The grounding contract has to hold while the wiring is wrong.